

Decisiones Arquitectónicas (ADRs) — Polizing-Interno

Entregable 4 · Práctica de Arquitectura y Diseño de Sistemas 2026 Grupo 6 — Felder, Ducos, Ferraro, Fueyo, Kravetz, Echavarría

Índice

ID	Decisión	Categoría	Estado
ADR-001	División del backend en dos servicios desplegables independientes	Arquitectura del backend	Aceptada
ADR-002	Elección de base de datos principal para la capa de persistencia del panel	Persistencia de datos	Aceptada
ADR-003	Mecanismo de comunicación entre el chatbot y el panel	Comunicación entre componentes	Aceptada
ADR-004	Estrategia de despliegue de los servicios del sistema	Infraestructura / Despliegue	Aceptada
ADR-005	Selección del proveedor de mensajería para el canal con clientes finales	Integración externa	Aceptada

Cómo se relacionan

ADR-001 establece la división en dos servicios (`my-app` y `chatbot`). De ahí salen las demás decisiones:

- ADR-002 elige PostgreSQL para `my-app` ; el `chatbot` mantiene su propio store local consistente con la separación.
- ADR-003 define cómo se hablan los dos servicios (REST síncrono con `X-API-Key`).
- ADR-004 define dónde corre cada servicio (Vercel + PaaS de contenedores), consecuencia operativa de la división.
- ADR-005 fija el proveedor del canal con clientes, justificando la existencia del `chatbot` como servicio separado.

Cómo exportar a PDF

Los ADRs viven en Markdown para versionarse junto al código. Para entregar un PDF consolidado (siguiendo el patrón de Entrega 3 con `modelo-datos.md` + `mod_datos.pdf`):

Opción A — con `pandoc` (recomendada)

```
# Instalar pandoc en macOS:
brew install pandoc basicstex

# Desde Entregables/Entrega 4/:
pandoc ADRs/README.md ADRs/ADR-001*.md ADRs/ADR-002*.md ADRs/ADR-003*.md ADRs/ADR-004*.md ADRs/ADR-005*.md \
  -o ADRs.pdf \
  --pdf-engine=xelatex \
```

```
-V geometry:margin=2cm \  
-V mainfont="Helvetica" \  
--toc
```

Opción B — sin instalar nada

Abrir cada `.md` en VSCode → extensión "Markdown PDF" (yzane.markdown-pdf) → click derecho → "Markdown PDF: Export (pdf)".

Opción C — desde GitHub

Subir el directorio, abrir el README en GitHub, imprimir → "Guardar como PDF" desde el navegador.

ADR-001 — División del backend en dos servicios desplegables independientes

ID	Título	Estado	Fecha
ADR-001	División del backend en dos servicios desplegables independientes	Aceptada	27/05/2026

1. Contexto

Polizing tiene dos audiencias claramente distintas que entran al sistema por canales distintos:

- **Productores y administrativos** usan un panel web para gestionar clientes, pólizas, siniestros y pagos durante su jornada laboral. Es un CRUD pesado con formularios y reportes.
- **Clientes finales** interactúan exclusivamente por WhatsApp para pedir su tarjeta de circulación, cargar comprobantes de pago o reportar siniestros. No tienen interfaz web propia.

Los motivadores que obligan a tomar una decisión arquitectónica:

- **Motivador 1 — Mantenibilidad (RNF Entrega 1):** los dos canales evolucionan a ritmos diferentes. El equipo necesita poder cambiar el flujo del bot (agregar una opción de menú, ajustar un texto) sin tocar ni desplegar el panel web, y viceversa. La normativa aseguradora cambia con frecuencia y forzar un único pipeline ralentiza ambos lados.
- **Motivador 2 — Disponibilidad 24/7 (RNF Entrega 1):** el chatbot debe seguir respondiendo aunque el panel web esté caído en una ventana de mantenimiento o liberando una versión. Un siniestro reportado a las 3 AM no puede depender de que un deploy del panel haya terminado bien.
- **Motivador 3 — Restricciones de runtime:** la integración con WhatsApp Cloud API requiere mantener un webhook público estable, sesiones HTTP largas para envío de medios y un proceso con estado de conversación en memoria/BD local. El stack del panel (Next.js sobre serverless) no encaja con ese perfil; el stack del bot (FastAPI sobre proceso persistente) no encaja con el de un panel React server-rendered.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Monolito único en Next.js (chatbot embebido como API routes)	Un solo deploy, un solo repo, un solo lenguaje (TypeScript). Modelo de datos compartido sin duplicación.	El webhook de WhatsApp y el manejador de sesiones conversacionales no encajan bien en un runtime serverless: pierden contexto entre invocaciones y obligan a workarounds caros. Acopla los ciclos de release de panel y bot, contradiciendo el motivador 1.
Microservicios completos (un servicio por dominio: clientes, pólizas, siniestros, pagos, bot, notificaciones)	Máxima independencia de despliegue por dominio; cada servicio puede escalar y evolucionar solo.	Sobreingeniería para un equipo de 6 personas en un PoC académico. Multiplica overhead de infraestructura (descubrimiento, observabilidad, contratos entre servicios) sin un volumen que lo justifique.

Dos servicios separados: panel web + chatbot (elegida)	Cada servicio usa el stack que mejor le sirve, despliegues independientes, fallo aislado por canal.	— (esta fue la elegida)
--	---	-------------------------

3. Decisión tomada

Se decide: dividir el backend en **dos servicios desplegables independientes** — `my-app` (panel web Next.js + API REST de negocio) y `chatbot` (servicio FastAPI que orquesta WhatsApp)— que se comunican entre sí únicamente por HTTP. Cada uno vive en su propio directorio del monorepo, con su propio stack, sus propias dependencias y su propio pipeline.

Fundamentación:

- Resuelve el motivador 1 (mantenibilidad y ritmo de evolución):** un cambio de copy en el bot o el agregado de una opción del menú se libera sin tocar `my-app`. Un refactor del modelo de siniestros en el panel no requiere redespargar el bot. Los equipos de cada lado pueden trabajar en paralelo.
- Resuelve el motivador 2 (disponibilidad 24/7):** si una versión del panel sale rota, el chatbot sigue tomando mensajes y encolando comprobantes/siniestros que se materializan en `my-app` cuando vuelve. La inversa también vale: si el bot cae, el panel sigue operativo para el productor.
- Resuelve el motivador 3 (runtimes incompatibles):** Next.js sobre Vercel resuelve bien el panel (server components, preview deploys, edge caché), mientras que FastAPI sobre un proceso persistente resuelve bien el bot (webhook estable, estado conversacional en BD local, librerías Python para integraciones futuras). Forzar un único runtime hubiera obligado a comprometer uno de los dos.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Cada servicio elige su stack óptimo (Next.js + Prisma para el panel; FastAPI + SQLAlchemy para el bot).	Aparece acoplamiento por contrato HTTP : cualquier cambio en endpoints expuestos por <code>my-app</code> que el bot consume rompe al bot si no se versiona la API.
Despliegues independientes: el ritmo del bot no está atado al ritmo del panel.	El modelo de dominio se duplica parcialmente (el bot necesita su propia representación de cliente, póliza, siniestro en SQLAlchemy, espejo de la del panel).
Aislamiento de fallos por canal: un bug en el panel no tira la atención al cliente final.	Dos pipelines de CI/CD, dos paneles de observabilidad, dos lugares donde rotar secretos (ver ADR-004).
Encaja con la estructura actual del repo (<code>/my-app</code> , <code>/chatbot</code>) sin forzar reorganización.	Latencia de red en cada interacción bot → panel (ver ADR-003 para cómo se mitiga).

Decisiones relacionadas:

- ADR-002 (persistencia en PostgreSQL): aplica al panel; el bot mantiene su propio store local consistente con esta separación.

- ADR-003 (comunicación REST síncrono con API Key): es la consecuencia directa de tener dos servicios que necesitan hablarse.
- ADR-004 (despliegue heterogéneo): los runtimes distintos obligan a infraestructura distinta para cada servicio.

ADR-002 — Elección de base de datos principal para la capa de persistencia del panel

ID	Título	Estado	Fecha
ADR-002	Elección de base de datos principal para la capa de persistencia del panel	Aceptada	27/05/2026

1. Contexto

El panel de Polizing (`my-app`) gestiona el ciclo de vida completo de productos de seguros: clientes (corporativos y personas), aseguradoras, pólizas, siniestros con documentos adjuntos, y pagos que agrupan varias pólizas en un mismo batch. El modelo de datos quedó documentado en `Entregables/Entrega 3/modelo-datos.md` (10 tablas, relaciones 1:N reales sin tablas puente innecesarias, enums para estados cerrados).

Motivadores que obligan a tomar una decisión sobre el motor de persistencia:

- **Motivador 1 — Modelo fuertemente relacional:** las entidades centrales (`clientes` , `polizas` , `siniestros` , `pagos` , `empresas_aseguradoras`) tienen relaciones 1:N reales y duras. Las consultas más frecuentes del panel requieren joins: listar pólizas de un cliente con su aseguradora y su próxima fecha de vencimiento; ver siniestros con todos sus documentos y la póliza asociada; reportes de pagos agrupados por cliente.
- **Motivador 2 — Consistencia transaccional (RNF Trazabilidad e Integridad, Entrega 1):** ciertas operaciones tocan varias tablas en un mismo flujo —dar de alta un siniestro crea el registro en `siniestros` y N registros en `siniestro_documentos` en una sola transacción; validar un pago actualiza `pagos` y propaga estado a las pólizas vinculadas. Si una falla, ninguna debe quedar a medias.
- **Motivador 3 — Trazabilidad por actor:** el modelo agrega `modificado_por_id` y `modificado_en` en cada tabla de negocio (decisión 1.6 del modelo de datos). Esto exige FKs con `SET NULL` , índices y constraints que un motor relacional resuelve nativamente.
- **Motivador 4 — Experiencia del equipo:** los 6 integrantes tienen experiencia con SQL y con Prisma (el proyecto ya nació con `prisma/schema.prisma`). Ninguno tiene experiencia productiva con NoSQL en sistemas con dominio relacional como este.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
MongoDB (NoSQL documental)	Schema flexible, ideal si las entidades fueran agregados autocontenidos. Buen rendimiento en lectura por documento.	El dominio es relacional, no documental : un cliente no contiene sus pólizas, pertenecen a una entidad distinta con su propio ciclo. Implementar transacciones multi-documento y joins en aplicación añade complejidad e impide expresar los constraints temporales (1.10 del modelo). Contradice motivadores 1 y 2.

MySQL	Ampliamente conocido, soporta transacciones ACID, buena comunidad.	Soporte más débil para tipos avanzados que el modelo usa o usará pronto (enum nativo limitado, JSONB, CHECK con expresiones, Decimal(15,2) con casts predecibles). Menos cómodo para evolucionar el schema con migraciones complejas. No aporta nada relevante por encima de la alternativa elegida en este caso.
PostgreSQL administrado por Supabase + Prisma ORM (elegida)	ACID completo, joins nativos, enums, decimales precisos, índices compuestos, CHECK constraints SQL, JSONB para documentos; Supabase elimina la operación del motor y aporta connection pooling listo.	— (esta fue la elegida)

3. Decisión tomada

Se decide: usar **PostgreSQL administrado por Supabase** como base de datos única del panel `my-app`, accedido mediante **Prisma ORM**. El schema final es el descrito en `Entregables/Entrega 3/modelo-datos.md` y materializado en `my-app/prisma/schema.prisma`.

Fundamentación:

- Resuelve el motivador 1 (modelo relacional):** PostgreSQL ejecuta nativamente todos los joins que el panel necesita para listados y reportes (clientes con sus pólizas vigentes, siniestros con su póliza y aseguradora, pagos con las pólizas agrupadas). No hay que reescribir esa lógica en código aplicativo como pasaría con un store documental.
- Resuelve los motivadores 2 y 3 (consistencia y trazabilidad):** las transacciones ACID permiten que crear un siniestro con sus N documentos sea atómico, y los CHECK constraints (`fecha_fin_vigencia >= fecha_inicio_vigencia` , `fecha_reporte >= fecha_ocurrencia`) protegen invariantes a nivel motor. Los campos `modificado_por_id` / `modificado_en` con FKs `SET NULL` se expresan naturalmente.
- Resuelve el motivador 4 (experiencia):** el equipo ya escribió `prisma/schema.prisma`, hizo el seed y las queries en `lib/data/*`. Cambiar de motor invalidaría todo ese trabajo y agregaría riesgo de migración en una entrega ya avanzada. Prisma además da migraciones declarativas y tipos generados que reducen errores en compile-time.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Los listados, KPIs y reportes del panel se implementan en SQL/Prisma directo, sin lógica adicional para emular joins.	Rigidez de schema: cualquier cambio del modelo exige una migración Prisma explícita (<code>prisma migrate dev</code>). Iterar sobre el modelo agrega fricción comparado con un store schemaless.
La consistencia transaccional la garantiza el motor: el flujo de carga de siniestro +	Vendor lock-in parcial con Supabase: se usan dos URLs (pooling y directa) y storage gestionado. Migrar a

documentos es atómico sin lógica de compensación.	otro proveedor de Postgres requeriría reconfigurar variables y revisar uso de extensiones.
Prisma genera tipos TypeScript a partir del schema, eliminando una clase entera de errores de runtime.	Escalabilidad horizontal limitada: Postgres escala mejor verticalmente. Si el volumen creciera mucho más allá del esperado para un productor de seguros mediano, se necesitaría sharding o réplicas de lectura.
Connection pooling de Supabase listo desde el día uno, alineado con el runtime serverless del panel (ver ADR-004).	El chatbot (ADR-001) no comparte esta base por estar en otro servicio; mantiene su propio store local. Hay duplicación parcial de modelo que el contrato REST (ADR-003) debe mantener consistente.

ADR-003 — Mecanismo de comunicación entre el chatbot y el panel

ID	Título	Estado	Fecha
ADR-003	Mecanismo de comunicación entre el chatbot y el panel	Aceptada	27/05/2026

1. Contexto

Como consecuencia de ADR-001, el sistema queda dividido en dos servicios (`my-app` y `chatbot`) que igual deben colaborar: cuando un cliente escribe por WhatsApp, el bot necesita validar que está registrado, listarle sus pólizas, descargar su tarjeta de circulación o crear un siniestro con documentos en el sistema principal. Las rutas concretas ya están prototipadas en `propuesta-de-avance/Usos-routes.md` (validación por teléfono, listar pólizas, solicitudes nuevas, tarjeta, comprobantes, siniestros).

Motivadores que obligan a tomar una decisión sobre el mecanismo de comunicación:

- **Motivador 1 — Patrón de interacción request/response del bot:** cada mensaje entrante del cliente exige una respuesta del bot en segundos (la conversación de WhatsApp es síncrona desde la perspectiva del usuario). El bot necesita el dato (¿este cliente existe?, ¿qué pólizas tiene?) antes de poder responder; no puede contestar y "ya te aviso".
- **Motivador 2 — Volumen acotado por la cadencia humana:** el chatbot atiende a clientes finales escribiendo a mano; el volumen pico realista está muy por debajo del que justificaría una capa de mensajería. No hay procesos batch ni fan-out masivo entre los servicios.
- **Motivador 3 — Seguridad (RNF Confidencialidad e Integridad, Entrega 1):** la API expuesta por `my-app` maneja datos sensibles (DNIs, pólizas, comprobantes de pago). Solo el chatbot —no usuarios anónimos— debe poder consumirla. Se necesita autenticación server-to-server simple y verificable.
- **Motivador 4 — Capacidad del equipo y plazo del PoC:** un equipo de 6 integrantes en un trabajo académico no puede operar broker de mensajes, esquemas Avro/Protobuf, dead-letter queues ni monitoreo de colas además de los dos servicios y la integración WhatsApp.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Cola de mensajes (RabbitMQ / SQS / Redis Streams)	Desacopla temporalmente bot y panel; si <code>my-app</code> cae, los mensajes se acumulan y se procesan al volver. Buena para picos de carga.	No encaja con el motivador 1: el bot necesita respuesta sincrónica para contestar al cliente (no puede esperar a que la cola procese y vuelva). Suma infraestructura (broker, workers, observabilidad) sin justificación de volumen (motivador 4).

Eventos / webhooks bidireccionales (pub-sub)	Cada servicio publica eventos de dominio; el otro reacciona. Útil para arquitecturas event-driven con muchos consumidores.	Sobrediseño para dos servicios con un patrón petición/respuesta puro. Complica el debug (la causa-efecto deja de ser lineal) y exige idempotencia y deduplicación que no son problemas reales acá.
REST síncrono server-to-server con API Key (elegida)	Petición/respuesta directa, fácil de razonar, fácil de testear con curl y mocks. La autenticación por header X-API-Key da control de acceso simple y suficiente para tráfico server-to-server.	— (esta fue la elegida)

3. Decisión tomada

Se decide: la comunicación entre el chatbot y el panel será **REST síncrono server-to-server sobre HTTPS**, con autenticación por **header X-API-Key** verificado en cada request de `my-app`. Los endpoints son los enumerados en `propuesta-de-avance/Usos-routes.md` (`GET /api/clients/by-phone/[phone]`, `GET /api/policies?phone=`, `POST /api/policy-requests`, `POST /api/circulation-card`, `POST /api/payment-receipts`, `POST /api/claims`). En el chatbot, la llamada está encapsulada en `chatbot/app/main_system_client.py`, con modo `mock` para desarrollo.

Fundamentación:

- Resuelve el motivador 1 (interacción síncrona):** el bot llama, recibe el dato y responde al cliente en el mismo turno. No hay que orquestar callbacks ni mantener estado intermedio entre llamada y respuesta.
- Resuelve los motivadores 2 y 4 (volumen y capacidad del equipo):** REST sobre HTTPS es el camino más simple y más debugueable; cualquier integrante del grupo lo entiende sin onboarding adicional. No agrega componentes nuevos a operar (broker, workers).
- Resuelve el motivador 3 (seguridad server-to-server):** el header `X-API-Key` es simple de implementar (validar en un middleware `Next.js`), permite rotar el secreto sin tocar usuarios finales y es suficiente para autenticar a un solo cliente conocido (el chatbot). El patrón ya está en uso en `chatbot/app/main_system_client.py`.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
El contrato es explícito y versionable (los endpoints están enumerados en <code>Usos-routes.md</code>).	Acoplamiento temporal: si <code>my-app</code> cae o tiene latencia alta, el bot no puede responder. El cliente percibe el caído como un caído del bot, no del panel.
Debug y testing simples: <code>curl</code> , Postman, mocks por archivo (<code>mock_system.py</code> ya existe en el chatbot).	Sin retry / idempotencia automática: una pérdida de paquete en <code>POST /api/claims</code> puede crear el siniestro y perder la confirmación, o no crearlo y dejar al cliente sin respuesta. Hay que tratarlo en cada handler.

API Key permite cortar acceso del bot inmediatamente si se compromete (rotar el secreto).	Sin event log nativo: no queda historial automático de "qué le pidió el bot al panel". Si se necesita auditar interacciones, hay que loggear explícitamente en ambos lados.
No agrega infraestructura nueva (sin broker, sin colas, sin schema registry).	El bot debe conocer la URL pública del panel (variable de entorno MAIN_SYSTEM_BASE_URL). En despliegues con preview environments hay que poblar esa variable por entorno.

Decisiones relacionadas:

- ADR-001 (separación de servicios): este ADR resuelve cómo se hablan los dos servicios separados.
- ADR-004 (despliegue heterogéneo): el panel debe exponer URL pública estable para que el chatbot pueda apuntarle.
- ADR-005 (WhatsApp Cloud API): el patrón síncrono encaja con el flujo entrante de WhatsApp, también request/response.

ADR-004 — Estrategia de despliegue de los servicios del sistema

ID	Título	Estado	Fecha
ADR-004	Estrategia de despliegue de los servicios del sistema	Aceptada	27/05/2026

1. Contexto

La división en dos servicios definida en ADR-001 obliga a decidir cómo y dónde se despliega cada uno. Los dos servicios tienen perfiles operativos distintos:

- `my-app` es Next.js 16 con App Router, Server Actions y Prisma sobre Postgres. Su carga es interactiva (sesiones de productor durante horario laboral), con picos cortos y mucho tiempo ocioso entre acciones.
- `chatbot` es FastAPI sobre Python 3.13 con SQLite local, un webhook público que Meta llama cada vez que un cliente escribe, y estado conversacional que vive en BD local entre turnos.

Motivadores que obligan a tomar una decisión arquitectónica de despliegue:

- Motivador 1 — Disponibilidad 24/7 (RNF Entrega 1):** el bot debe poder ser invocado por Meta en cualquier momento; la URL pública debe ser estable y certificada (HTTPS). El panel debe estar disponible al menos en horario extendido para productores.
- Motivador 2 — Restricciones técnicas opuestas entre los dos servicios:** el panel encaja muy bien en plataformas serverless por su perfil bursty y por la integración nativa Next.js + Vercel + Supabase. El bot, en cambio, **no** encaja en serverless: tiene BD local (SQLite en `/data/chatbot.db`), webhook con sesión HTTPS, y un estado conversacional que se perdería entre invocaciones efímeras.
- Motivador 3 — Capacidad del equipo (sin DevOps dedicado):** ningún integrante administra clusters de Kubernetes ni VMs en producción. La operación tiene que reducirse al mínimo: configurar, conectar el repo y olvidar.
- Motivador 4 — Costo:** proyecto académico/PoC, sin presupuesto para infraestructura on-premise ni planes empresariales.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Todo en una VPS con Docker Compose (un VPS aloja panel + bot + Postgres)	Un solo lugar a operar, costos predecibles, control total.	Contradice los motivadores 1 y 3: cualquier reinicio de la VM tira los dos servicios a la vez; requiere que alguien del equipo opere la VM (backups, certificados TLS, monitoring). Pierde la integración nativa <code>Next.js↔Vercel↔Supabase</code> .
Todo en Kubernetes (clúster propio o gestionado)	Despliegues canarios, escalado horizontal, networking sofisticado.	Sobreingeniería absoluta para dos servicios y un equipo de 6 personas. Curva de operación demasiado pronunciada (motivador 3); el costo de un clúster gestionado es alto (motivador 4).

Vercel para el panel + PaaS de contenedores para el bot (Fly.io / Railway / Render) (elegida)	Cada servicio en la plataforma que mejor le sirve; ambas plataformas se gestionan por dashboard y conectan al repo de Git. Cero servidores que administrar.	— (esta fue la elegida)
---	---	-------------------------

3. Decisión tomada

Se decide: desplegar `my-app` en **Vercel** (con la base de datos PostgreSQL administrada por Supabase, ver ADR-002) y `chatbot` como contenedor en un **PaaS de contenedores** (Fly.io o Railway), expuesto vía URL pública con HTTPS provista por la plataforma. Cada servicio tiene su propio pipeline conectado al monorepo (filtrado por path).

Fundamentación:

- Resuelve el motivador 1 (disponibilidad y URL pública estable):** Vercel da TLS y URL estable para el panel automáticamente; el PaaS de contenedores da una URL `https://...` estable que se registra como webhook en Meta. Ambos servicios escalan/recuperan sin intervención manual.
- Resuelve el motivador 2 (restricciones técnicas opuestas):** Vercel está hecho para Next.js (server components, edge caché, preview deploys por PR) y se integra nativamente con Supabase. Un PaaS de contenedores corre el bot como proceso persistente con volumen para `/data/chatbot.db`, preservando estado entre requests; ningún integrante toca un Dockerfile complejo.
- Resuelve los motivadores 3 y 4 (sin DevOps, costo bajo):** ambas plataformas tienen tier gratuito o muy barato suficiente para un PoC; la operación se reduce a configurar variables de entorno y revisar logs en el dashboard. Cero servidores propios, cero certificados manuales.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Cada servicio en la plataforma que mejor encaja con su runtime: deploys rápidos en ambos lados.	Dos paneles de operación distintos (Vercel + PaaS de contenedores). Las métricas, logs y alertas no están unificadas.
Preview deploys del panel en cada PR sin trabajo adicional (Vercel nativo).	Los secretos (URLs de Supabase, X-API-Key de ADR-003, credenciales de Meta de ADR-005) se gestionan en dos lugares , con riesgo de divergencia entre entornos.
Vendor de hosting está atado al stack, pero ambos son sustituibles: el bot vive en un contenedor estándar y el panel es Next.js puro.	El bot mantiene estado local en SQLite : si el contenedor se mueve de nodo sin volumen persistente, se pierde el estado conversacional en curso. Hay que asegurar configuración de volumen en la plataforma elegida.
Disponibilidad por servicio: caída del bot no afecta al panel y viceversa, alineado con ADR-001 y ADR-003.	Latencia de red entre el bot y el panel: las llamadas REST cruzan internet entre dos PaaS distintos. Tolerable para el patrón request/response definido en ADR-003, pero hay que medirla.

Decisiones relacionadas:

- ADR-001 (separación de servicios): este ADR es la materialización operativa de aquella decisión.
- ADR-002 (PostgreSQL en Supabase): la elección del proveedor de BD encaja con Vercel.
- ADR-003 (REST con API Key): el bot necesita conocer la URL pública del panel; esa URL la provee Vercel.

ADR-005 — Selección del proveedor de mensajería para el canal con clientes finales

ID	Título	Estado	Fecha
ADR-005	Selección del proveedor de mensajería para el canal con clientes finales	Aceptada	27/05/2026

1. Contexto

El chatbot definido en ADR-001 atiende a clientes finales a través de un canal de mensajería. La interacción incluye texto bidireccional, recepción de imágenes y PDFs (comprobantes de pago, fotos de siniestros, actas policiales) y envío de documentos (tarjeta de circulación en PDF). Para producir esos flujos se necesita elegir un proveedor concreto que conecte el servicio con la red de mensajería que ya usan los clientes.

Motivadores que obligan a tomar una decisión:

- **Motivador 1 — El canal lo define el usuario:** los clientes finales de productores de seguros en Argentina ya tienen WhatsApp instalado y lo usan a diario. Pedirles que instalen una app nueva o usen un canal alternativo (SMS, email) baja drásticamente la adopción. La elección de canal no es libre: es WhatsApp.
- **Motivador 2 — Necesidades técnicas concretas:** se necesitan templates aprobados para mensajes iniciados por el negocio (notificaciones de vencimiento), webhooks confiables para entrada (cada mensaje del cliente llega al bot) y APIs de envío programático con soporte de medios (imágenes, PDFs).
- **Motivador 3 — Costo (PoC académico):** el sistema debe poder operar en un tier gratuito o muy barato durante el desarrollo y la presentación del trabajo. No hay presupuesto para licenciamiento enterprise ni para fees per-message altos.
- **Motivador 4 — Manejo de fallos y rate limits (RNF Interoperabilidad, Entrega 1):** el sistema debe estar preparado para que el proveedor aplique throttling, rechace mensajes o tenga ventanas de mantenimiento, sin que esto rompa la conversación con el cliente.

2. Alternativas consideradas

Alternativa	Ventaja principal en este contexto	Desventaja / Motivo de descarte
Twilio WhatsApp API (agregador)	SDK maduro, documentación excelente, soporte 24/7, sandbox de pruebas inmediato. Abstrae a Meta.	Costo por mensaje elevado (markup sobre la tarifa de Meta), contradice motivador 3. La capa de abstracción agrega latencia y oculta features que Meta libera primero en su API directa.
360dialog (BSP — Business Solution Provider)	Acceso oficial WhatsApp Business API con soporte comercial; precio más competitivo que Twilio.	Requiere onboarding comercial (verificación de empresa) y un mínimo de fee mensual no compatible con un PoC académico (motivador 3). Sin sandbox abierto inmediato.

Meta WhatsApp Cloud API (directa, elegida)	Acceso oficial directo al canal, tier de mensajes de servicio gratuito muy generoso , números de prueba inmediatos, control total sobre templates y números.	— (esta fue la elegida)
--	---	-------------------------

3. Decisión tomada

Se decide: integrar el chatbot directamente con **Meta WhatsApp Cloud API**, encapsulando la llamada en `chatbot/app/whatsapp.py` (cliente HTTPS para envío) y exponiendo el webhook de entrada en `chatbot/app/api.py` (GET/POST /webhooks/whatsapp). Los templates iniciados por negocio se registran y aprueban directamente en el WhatsApp Business Manager de Meta.

Fundamentación:

- 1. Resuelve los motivadores 1 y 3 (canal correcto y costo):** Meta es la fuente oficial; ir directo evita el markup de los agregadores. El tier gratuito de Cloud API alcanza para el volumen del PoC y para la demo de la cátedra sin requerir tarjeta de crédito.
- 2. Resuelve el motivador 2 (necesidades técnicas):** Cloud API provee templates, webhooks de entrada y endpoints de envío de medios (imágenes y PDFs) usados en los flujos de comprobante y de tarjeta de circulación. Estas features están disponibles desde el día uno, sin esperar a que un agregador las exponga.
- 3. Aislamiento del proveedor:** el cliente HTTP vive solo en `chatbot/app/whatsapp.py`. Si en el futuro hubiera que migrar a un BSP (por necesidad de soporte enterprise o por SLA), el cambio queda contenido en ese módulo. La decisión es reversible al costo de reescribir un archivo.

4. Consecuencias

Consecuencias positivas	Trade-offs / costos
Costo cero/marginal durante PoC y demo.	Gestión propia de templates y aprobaciones: cada mensaje iniciado por el negocio requiere registrar un template y esperar la aprobación de Meta (puede tardar horas o ser rechazado).
Acceso a features nuevas en cuanto Meta las publica, sin esperar a un intermediario.	Sin SLA premium: ante incidentes de Meta no hay soporte humano dedicado, solo status page y documentación pública. Hay que tolerar ventanas ocasionales (motivador 4).
El cliente HTTP encapsulado en <code>chatbot/app/whatsapp.py</code> mantiene el resto del bot agnóstico al proveedor.	Rate limits sin smoothing automático: el bot debe respetar los límites de Meta por número (mensajes/segundo, calidad del número). En el PoC el riesgo es bajo, pero en producción real exigiría una capa de rate limiting/cola interna.
Verificación de webhook estándar (HMAC contra <code>APP_SECRET</code>), encaja con el patrón de seguridad de ADR-003.	Acoplamiento al proveedor sigue existiendo: los formatos de webhook y de payload son específicos de Meta; migrar implica reescribir el handler de entrada, no solo el cliente de salida.

Decisiones relacionadas:

- ADR-001 (separación de servicios): el chatbot existe como servicio separado, en gran parte por la necesidad de hostear este webhook estable.
- ADR-003 (REST con API Key): el patrón request/response sincrónico del bot encaja con el flujo entrante de WhatsApp.
- ADR-004 (despliegue): el PaaS de contenedores provee la URL pública estable que Meta requiere como webhook.